SYSTEM ARCHITECTURE DIAGRAM · MAY 2026 · V1.0 · COMPANION TO THE TECHNICAL WHITE PAPER

Bearcast Media
System Architecture

A full-fidelity map of every actor, service, and data flow that produces bearcastmedia.com.

The technical white paper described this architecture in prose. This document presents the same architecture visually, in greater detail, at a scale large enough to study. Each diagram is a complete and accurate representation of how the live system operates as of May 2026.

The four diagrams progress from high to low level: **(1)** the complete system overview showing every actor and every service; **(2)** the editorial workflow from a writer's keystrokes to a published article; **(3)** the public visitor's request flow with cache layers, asset origins, and runtime fetches; and **(4)** the radio pipeline showing how live audio, station metadata, and the CMS-published schedule converge.

COMPANION TO
Bearcast Technical White Paper
v1.0 · May 2026

AUTHORED BY
Cam Garrison
Web Director

LAST UPDATED
May 2026

Reading This Document

Node Types

Actor / Person

Actors. A human role that interacts with the system — editors, directors, visitors, DJs, the Web Director.
`Orange · Rounded rectangle`

External Service

External services. Third-party platforms Bearcast depends on — Sanity, Cloudflare, Radio.co, GitHub.
`Blue · Hard-cornered rectangle`

Bearcast Asset

Bearcast assets. First-party code, content, and configuration that Bearcast owns and maintains — the site code, the Sanity dataset, the Studio.
`Red · Hard-cornered rectangle`

CDN / Edge

Edge / CDN layer. Geographically distributed cache and serving infrastructure operated by Cloudflare and Sanity.
`Green · Stadium shape`

Data Artifact

Data artifacts. Concrete data shapes that flow over the wire — a JSON response, an audio stream, a webhook payload.
`Purple · Dashed border`

Edge Types

Read / Request. Data flowing one way — a GET, a fetch, a download.

Write / Mutation. A change being committed — a Sanity publish, a git push, a deploy.

Async / triggered. Event-driven — a webhook, an auto-deploy, a background sync.

Audio stream. Live audio data flowing continuously — DJ broadcast, listener stream.

Edge Labels

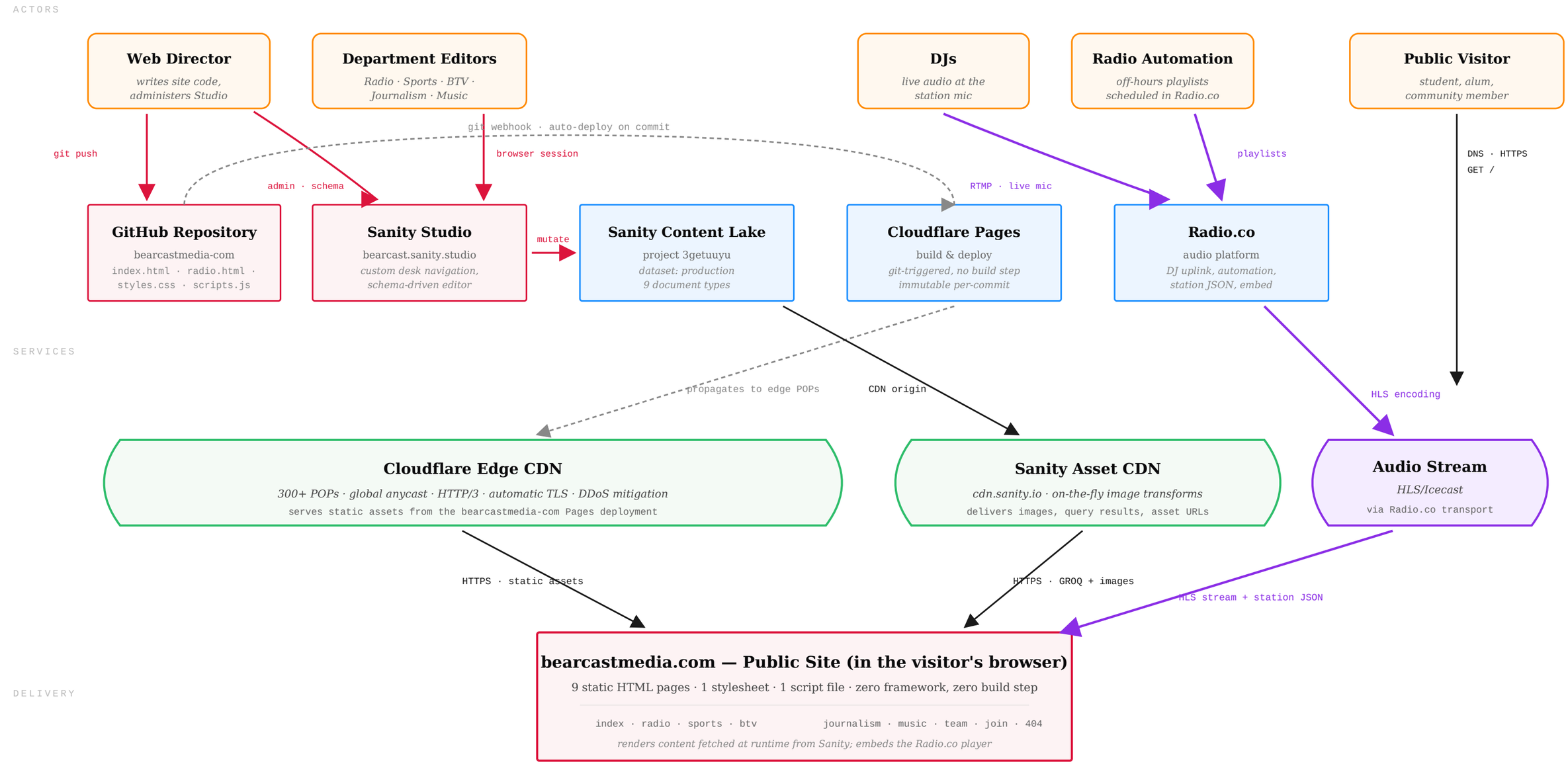
Every edge in these diagrams is labeled with the *protocol* and the *payload*. For example:

- `HTTPS · GROQ query` — an HTTPS request carrying a GROQ-syntax query string
- `HTTPS · JSON` — an HTTPS response carrying JSON
- `HTTPS · webhook` — a webhook payload over HTTPS
- `HLS · Opus/AAC` — an HTTP Live Streaming audio stream

Reading the labels is how you understand *what* is moving, not just *that* something is moving.

Diagram 1 — Full System Overview

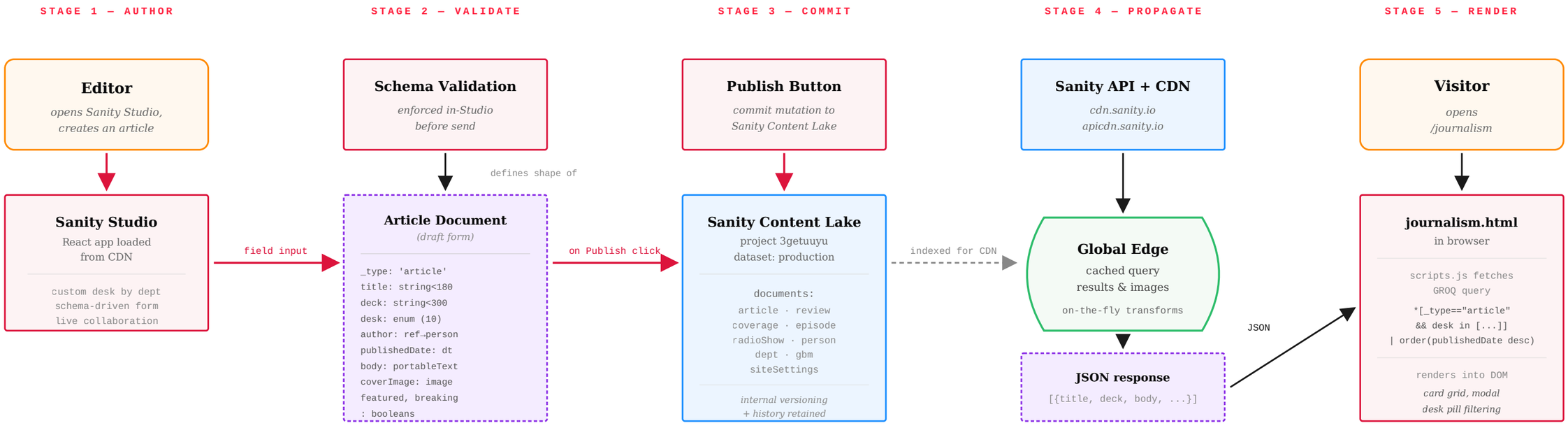
EVERY ACTOR, EVERY SERVICE, EVERY FLOW



How to read this diagram. The top row contains human actors. The middle row contains the services those actors interact with. The green and purple stadium shapes are edge / streaming layers — the geographically distributed infrastructure that actually serves data to visitors. The red box at the bottom is the single convergence point: the public site running in the visitor's browser, which is where every other component eventually delivers something. Red arrows are mutations (writes); black arrows are reads; dashed gray arrows are async / event-driven; purple is live audio. Read the arrow labels to understand *what* is flowing, not just *that* something is flowing.

Diagram 2 — Editorial Workflow

FROM A WRITER'S KEYSTROKES TO A PUBLISHED ARTICLE



~minutes	~instant	~< 1 sec	~seconds	~live
human writing time	client-side check	Sanity API call	edge cache fill	next visitor sees update

Why this design produces fast turnaround:

1. Editor input is captured by a schema-driven form, so the Studio rejects invalid documents before they ever reach the network. Type errors, missing required fields, and validation failures (e.g. a 12-character YouTube ID) are surfaced inline; the editor cannot "publish broken."

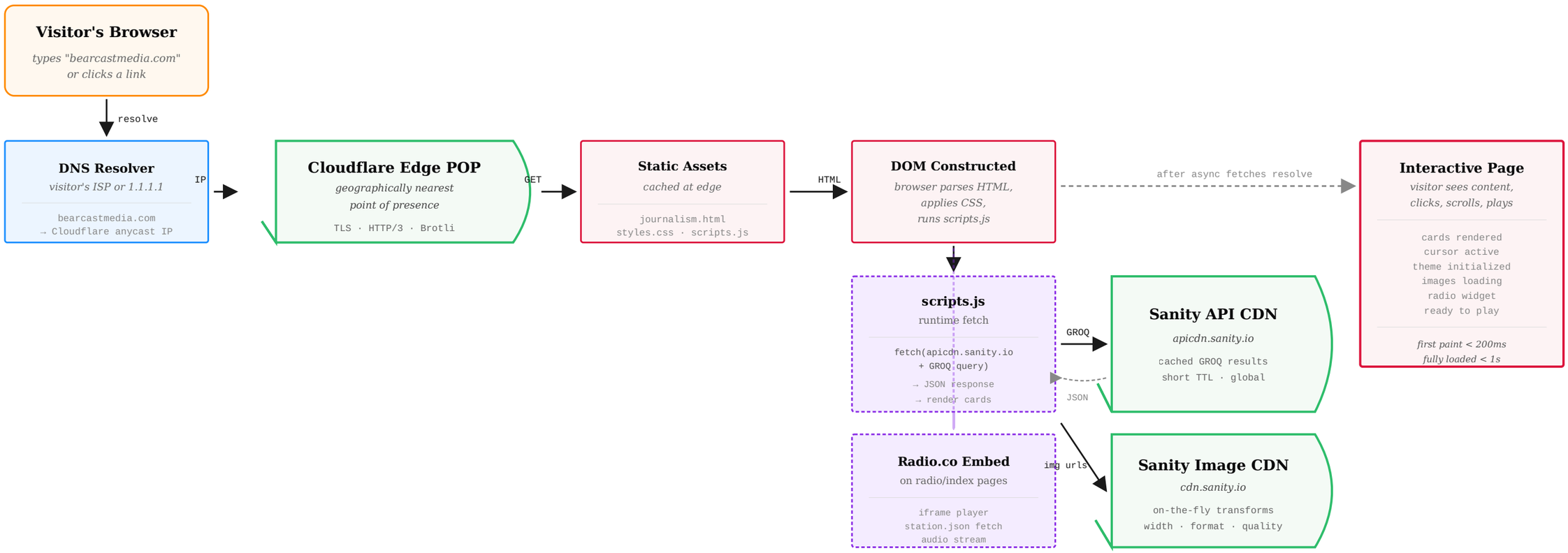
2. On Publish, the Studio issues a mutation to the Content Lake over Sanity's API. No build pipeline runs. No site rebuild is triggered. The public site does not deploy. The change is live in the data layer in under a second.

3. The next visitor to load `journalism.html` the article the editor just published. Edge cache TTLs are short for queries, so the new article surfaces within seconds, not minutes.

4. Crucially: editors never touch code, never wait for a build, and never see deploy logs. The architectural separation between content and code is what makes this workflow safe to give to non-technical editors.

Diagram 3 — Visitor Request Flow

WHAT HAPPENS WHEN SOMEBODY OPENS BEARCASTMEDIA.COM



~10–40 ms	~30–80 ms	~30–60 ms	~100 ms	~80–150 ms	< 1 s total
DNS lookup	TLS + edge connect	asset transfer	parse + first paint	runtime data fetch	fully interactive

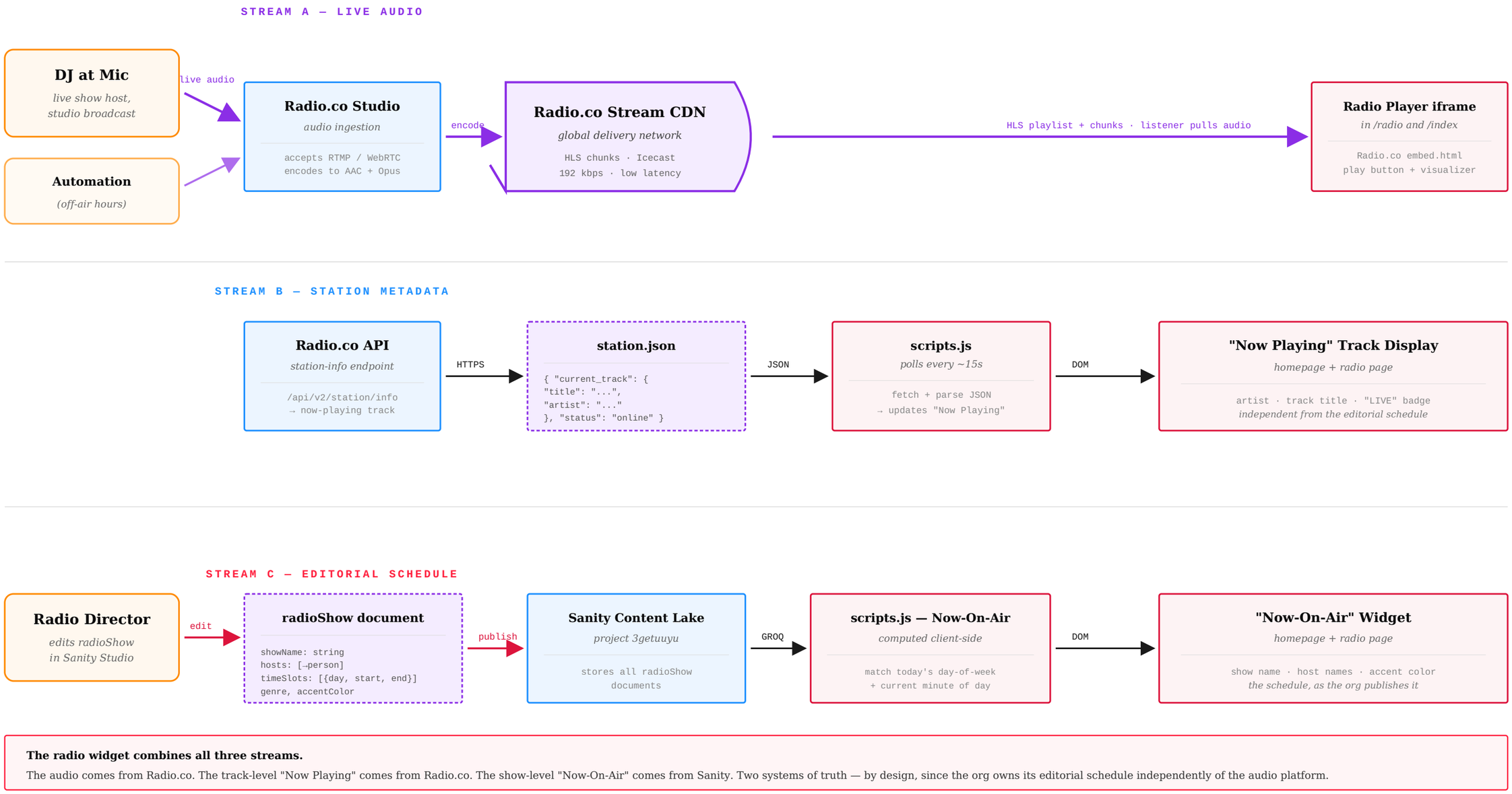
Architectural observations from this request flow:

▸ The HTML, CSS, and JS hit the visitor from the nearest Cloudflare POP — the same files everyone everywhere gets. They are cached aggressively (immutable per-deploy assets) and are typically served in under 100ms anywhere in the world.

▸ The content fetch is a separate round-trip that happens after first paint. The page is visually framed before the data arrives, then progressively fills in. This is deliberate — the user-perceived load is dominated by the static shell, not by the data round-trip.

Diagram 4 — Radio Pipeline

HOW LIVE AUDIO, STATION METADATA, AND THE CMS SCHEDULE CONVERGE IN THE RADIO WIDGET



The radio widget combines all three streams.

The audio comes from Radio.co. The track-level "Now Playing" comes from Radio.co. The show-level "Now-On-Air" comes from Sanity. Two systems of truth — by design, since the org owns its editorial schedule independently of the audio platform.

CLOSING

Reading These Diagrams Together

What the Four Diagrams Show Together

Diagram 1 is the map. It shows every actor, every service, every flow that produces bearcastmedia.com — at the highest level. Anyone trying to understand the system for the first time should start here.

Diagrams 2, 3, and 4 are the three most important workflows within that system, drawn in detail:

- **Diagram 2** follows a single article from an editor's first keystroke to the moment the next visitor reads it.
- **Diagram 3** follows a single page request from the moment somebody types `bearcastmedia.com` to the moment the page is fully interactive.
- **Diagram 4** follows the radio pipeline — the three independent streams (live audio, station metadata, editorial schedule) that meet inside a single widget.

What These Diagrams Do Not Show

The diagrams describe the operational architecture: how data moves, where it lives, who triggers what. They deliberately omit:

- **Internal Sanity / Cloudflare implementation.** Those vendors operate their own infrastructure; we use them as black boxes. The diagrams show what we send them and what we get back.
- **Code-level detail.** The exact JavaScript that fetches from Sanity, or the schema validation rules, are documented in the Technical White Paper.
- **Editorial workflow.** Who reviews what, who approves what, when a story moves from draft to published — those are organizational questions, documented in the Editor Handbook.

How to Use This Document

For a new Web Director: read Diagram 1 first to learn the terrain, then study Diagrams 2 and 3 to understand the two flows you'll touch daily.

For stakeholders and reviewers: Diagram 1 alone communicates the architecture. The other three diagrams provide the evidence that the architecture works as advertised.

For incident response: when something is broken, identify which diagram describes the affected flow, then trace each arrow from the symptom backward. The arrow labels tell you what should be moving — if something isn't, that arrow is the place to look.

Maintaining This Document

These diagrams are accurate as of May 2026 and should be updated whenever the architecture meaningfully changes. Specifically:

- Adding a new external service.
- Adding a new document type to the Sanity schema.
- Changing the deployment pipeline.
- Changing the radio integration.
- Restructuring the public site's page set.

Cosmetic changes (typography, copy, design polish) do not require updating these diagrams. The diagrams describe *what flows where*, not *what it looks like*.

Companion Documents

This diagram set is part of a four-document Bearcast Media documentation package:

- **Technical White Paper** — the prose treatment of this architecture, with code samples, design rationale, and comparison to the inherited system.
- **Editor Handbook** — how to use Sanity day-to-day, by department.
- **Maintenance Runbook** — how to keep the site alive across Web Director transitions.
- **System Architecture Diagram** (this document) — the visual map.

Bearcast Media System Architecture Diagram · v1.0 · May 2026
Authored by Cam Garrison · Companion to the Technical White Paper · For internal Bearcast Media use.